

PARCIAL DE UML, POO Y PATRONES DE DISEÑO

Tema: Sistema de gestión de FRANXX — Darling in the FranXX

Tiempo estimado: 90 a 120 minutos

Puntaje total: 100 puntos

SITUACIÓN PROBLEMÁTICA

La organización APE necesita desarrollar un sistema orientado a objetos para administrar los FRANXX utilizados por los distintos escuadrones de pilotos.

Cada FRANXX posee un identificador, un nombre, un estado operativo, un nivel de energía, un arma principal y una lista de pilotos compatibles.

El sistema debe contemplar diferentes modelos de FRANXX, entre ellos:

- Strelizia
- Delphinium
- Argentea
- Genista
- Chlorophytum

Cada modelo de FRANXX posee características y comportamientos particulares.

Los pilotos pertenecen a un escuadrón y poseen los siguientes datos:

- Nombre.
- Código.
- Edad.
- Nivel de sincronización.
- Estado físico.
- FRANXX compatibles.

Para operar un FRANXX normalmente se necesitan dos pilotos: un piloto masculino y una piloto femenina.

Durante una misión, un escuadrón puede recibir diferentes tipos de órdenes:

- Patrullar.
- Atacar un Klaxosaurio.
- Defender una plantación.
- Retirarse.

Los FRANXX también poseen diferentes estados operativos:

- Disponible.
- En misión.

- Dañado.
- Reparándose.

El sistema debe controlar las transiciones entre estos estados.

Además, se deben registrar las misiones realizadas y permitir consultar información relacionada con los pilotos y los FRANXX.

El sistema deberá ser diseñado utilizando conceptos de Programación Orientada a Objetos, UML y patrones de diseño.

CONSIGNA 1 — ANÁLISIS ORIENTADO A OBJETOS

20 puntos

A partir de la situación problemática planteada:

1. Identifique al menos **8 clases** necesarias para representar el sistema.
2. Para cada clase indique:
3. Nombre.
4. Atributos.
5. Métodos principales.
6. Responsabilidad dentro del sistema.
7. Determine cuáles de las clases identificadas deberían ser:
8. Clases concretas.
9. Clases abstractas.
10. Interfaces.
11. Identifique las relaciones existentes entre las clases.
12. Determine cuáles de las relaciones corresponden a:
13. Asociación.
14. Agregación.
15. Composición.
16. Herencia.
17. Dependencia.
18. Indique las multiplicidades correspondientes a las asociaciones.

19. Justifique brevemente las decisiones tomadas en el modelado.

CONSIGNA 2 — DIAGRAMA DE CLASES UML

25 puntos

Realice el **diagrama de clases UML completo** correspondiente al sistema.

El diagrama deberá incluir:

- Clases.
- Atributos.
- Métodos.
- Visibilidad de atributos y métodos.
- Relaciones.
- Multiplicidades.
- Herencia.
- Interfaces.
- Clases abstractas.
- Dependencias.
- Agregaciones.
- Composiciones.

El diseño deberá representar correctamente la relación entre:

- Escuadrones.
- Pilotos.
- FRANXX.
- Modelos concretos de FRANXX.
- Misiones.
- Órdenes de combate.
- Armas.
- Estados de los FRANXX.

Justifique las relaciones de composición y agregación utilizadas.

CONSIGNA 3 — PATRÓN STRATEGY

10 puntos

Las estrategias utilizadas durante una misión pueden variar entre:

- Ataque.
- Defensa.
- Patrulla.
- Retirada.

Utilice el patrón de diseño **Strategy** para modelar esta situación.

1. Identifique cuál será la clase que actuará como **Contexto**.
 2. Defina la interfaz o clase abstracta correspondiente a la estrategia.
 3. Defina las estrategias concretas.
 4. Represente las relaciones entre las clases mediante UML.
 5. Explique cómo un FRANXX podría cambiar de estrategia durante una misión sin modificar su clase principal.
 6. Explique qué problema tendría una solución basada únicamente en múltiples estructuras `if/else` o `switch`.
-

CONSIGNA 4 — PATRÓN FACTORY

10 puntos

APE desea crear diferentes tipos de FRANXX sin que el código principal tenga que conocer directamente las clases concretas.

Por ejemplo, se desea poder solicitar:

- Strelizia.
- Delphinium.
- Argentea.
- Genista.
- Chlorophytum.

Utilice el patrón **Factory** o **Factory Method** para resolver este problema.

1. Diseñe la fábrica correspondiente.
 2. Determine cuál será la clase o interfaz base de los FRANXX.
 3. Determine cuáles serán las clases concretas.
 4. Realice el diagrama UML correspondiente.
 5. Explique cómo se crearía un FRANXX a través de la fábrica.
 6. Justifique las ventajas de utilizar una fábrica frente a crear directamente los objetos mediante `new`.
-

CONSIGNA 5 — PATRÓN STATE

10 puntos

Los FRANXX pueden encontrarse en diferentes estados:

- Disponible.
- En misión.
- Dañado.
- Reparándose.

Un FRANXX debe comportarse de manera diferente dependiendo de su estado actual.

Por ejemplo:

- Un FRANXX disponible puede iniciar una misión.
- Un FRANXX en misión puede atacar.
- Un FRANXX dañado no puede iniciar una misión.
- Un FRANXX en reparación no puede combatir.
- Un FRANXX reparado puede volver a estar disponible.

Utilice el patrón **State** para modelar esta situación.

1. Defina la interfaz o clase abstracta `EstadoFranxx`.
 2. Defina los estados concretos.
 3. Determine qué operaciones puede realizar el FRANXX en cada estado.
 4. Realice el diagrama de clases UML correspondiente.
 5. Represente las principales transiciones entre estados.
 6. Explique por qué resulta conveniente utilizar State en lugar de colocar toda la lógica de estados dentro de una única clase `Franxx`.
-

CONSIGNA 6 — PATRÓN OBSERVER

10 puntos

Cuando un FRANXX cambia de estado, diferentes componentes del sistema deben ser notificados.

Los componentes interesados son:

- Centro de Comando.
- Sistema de Mantenimiento.
- Registro de Misiones.

Por ejemplo, cuando un FRANXX pasa de:

En misión → Dañado

el sistema debe notificar automáticamente a los componentes interesados.

Utilice el patrón **Observer** para resolver esta situación.

1. Identifique el objeto que actuará como **Subject/Observable**.
2. Identifique los objetos que actuarán como **Observers**.
3. Defina la interfaz correspondiente.
4. Realice el diagrama UML.
5. Explique cómo se produce la notificación cuando cambia el estado del FRANXX.
6. Explique qué ventaja ofrece este patrón respecto de hacer que `Franxx` conozca directamente a cada uno de los sistemas que deben recibir la notificación.

CONSIGNA 7 — PROGRAMACIÓN ORIENTADA A OBJETOS

10 puntos

Explique cómo se aplican los siguientes conceptos de POO dentro del sistema:

A. Encapsulamiento

Indique qué atributos deberían ser `private`, `protected` o `public` y justifique su elección.

B. Herencia

Proponga una jerarquía de clases para los diferentes modelos de FRANXX.

C. Abstracción

Determine qué comportamiento debería estar definido en una clase abstracta o interfaz común para todos los FRANXX.

D. Polimorfismo

Explique cómo podría ejecutarse:

```
franxx.ejecutarMision()
```

sin necesidad de conocer si el objeto corresponde a Strelizia, Delphinium, Argentea, Genista o Chlorophytum.

CONSIGNA 8 — DIAGRAMA DE SECUENCIA

10 puntos

Realice un **diagrama de secuencia UML** correspondiente al siguiente caso de uso:

"Un escuadrón recibe una orden para atacar a un Klaxosaurio."

El proceso deberá contemplar como mínimo la participación de:

- Centro de Comando.
- Escuadrón.
- Pilotos.
- FRANXX.
- Estrategia de combate.
- Klaxosaurio.

El diagrama deberá representar:

- Objetos.
- Mensajes.
- Llamadas a métodos.
- Retornos.
- Activaciones.
- Decisiones necesarias.

Además, deberá contemplar qué sucede si el FRANXX no se encuentra disponible para iniciar la misión.

CONSIGNA 9 — PRINCIPIOS SOLID

5 puntos

Analice el diseño realizado e indique qué principios SOLID se encuentran presentes.

Explique especialmente:

1. **Single Responsibility Principle (SRP).**
2. **Open/Closed Principle (OCP).**
3. **Dependency Inversion Principle (DIP).**

Justifique cada respuesta utilizando ejemplos concretos del sistema.

CONSIGNA 10 — SITUACIÓN INTEGRADORA

10 puntos

APE decide incorporar un nuevo modelo de FRANXX denominado:

RAIDER

Este nuevo modelo posee:

- Un arma especial.
- Una habilidad exclusiva.
- Una nueva estrategia de combate.
- Un comportamiento diferente durante determinadas misiones.

El nuevo FRANXX debe poder incorporarse al sistema sin realizar modificaciones importantes sobre las clases existentes.

Responda:

1. ¿Qué clases nuevas deberían incorporarse?
2. ¿Qué patrones de diseño existentes permitirían incorporar el nuevo FRANXX?
3. ¿Qué modificaciones serían necesarias en el sistema?
4. ¿Qué principio SOLID se favorece principalmente?
5. Explique cómo el diseño realizado permite extender el sistema sin modificar excesivamente el código existente.

CONSIGNA 11 — PREGUNTA DE FUNDAMENTACIÓN

El profesor plantea la siguiente situación:

"En lugar de utilizar el patrón Strategy, podríamos colocar un `switch` dentro de la clase `Franxx` para determinar si el FRANXX debe atacar, defender, patrullar o retirarse."

Explique:

1. ¿Qué problemas podría generar esta solución?
2. ¿Qué ventajas aporta Strategy?

3. ¿Cómo mejora la mantenibilidad del sistema?
 4. ¿Cómo facilita agregar una nueva estrategia en el futuro?
 5. ¿Qué principio SOLID se ve favorecido mediante esta decisión?
-

ENTREGA

El alumno deberá entregar:

1. Análisis de las clases.
2. Diagrama de clases UML.
3. Aplicación del patrón Strategy.
4. Aplicación del patrón Factory.
5. Aplicación del patrón State.
6. Aplicación del patrón Observer.
7. Explicación de los conceptos de POO.
8. Diagrama de secuencia.
9. Justificación de los principios SOLID utilizados.
10. Resolución de la situación integradora.
11. Fundamentación de las decisiones de diseño.

Todas las decisiones de diseño deberán estar **justificadas**.